Universidad Laica Eloy Alfaro de Manabí

Facultad ciencias de la vida y tecnología

Carrera: ING. Software

Tema:

Organización de la lógica de la API mediante controladores en Laravel



Autor:

Cardenas Ávila Emilio Sleimen

Docente:

Ing. Junior Zamora

Asignatura:

Integración e Implementación de Software

Paralelo:

“A”

Descripción y alcance del proyecto

NexoCommerce es una plataforma de comercio electrónico para un negocio de repostería, detalles y sublimación. Su propósito es organizar la consulta de novedades y productos, la personalización y el proceso de compra de un negocio gestionado mediante redes sociales y WhatsApp.

Separación de responsabilidades

routes/api.php declara las cuatro rutas y dirige cada solicitud al controlador correspondiente. No contiene funciones anónimas ni lógica de respuesta. El prefijo /api se registra en bootstrap/app.php. La lógica de consulta y cálculo está en los controladores.

PublicacionController

index devuelve el listado público de novedades, trabajos realizados y promociones. show recibe un ID numérico y devuelve una publicación; si el ID no existe, responde con HTTP 404 y un mensaje descriptivo.

CarritoController

index muestra los artículos del cliente autenticado, sus cantidades, precios unitarios y subtotales. resumen calcula productos distintos, cantidad total de unidades e importe total en USD. Los importes se suman en centavos y se convierten a dólares al responder; no se calculan impuestos, descuentos ni envío.

Ambos métodos obtienen al cliente desde la autenticación de Sanctum, no desde un ID enviado por el solicitante. Sin token válido responden 401. Un cliente sin artículos recibe una lista vacía y un resumen con ceros.

Rutas y comandos utilizados

GET /api/publicaciones

PublicacionController@index — Público

GET /api/publicaciones/{id}

PublicacionController@show — Público

GET /api/carrito

CarritoController@index — Token requerido

GET /api/carrito/resumen

CarritoController@resumen — Token requerido

Creación de los controladores

Los siguientes comandos se ejecutaron dentro de apilaravel. Laravel, PHP, Composer y Sanctum ya estaban instalados y se conservaron.

php artisan make:controller Api/PublicacionController --no-interaction

php artisan make:controller Api/CarritoController --no-interaction

Archivos de apoyo

Se añadió HasApiTokens al modelo User para autenticar con Sanctum. PrepararDemoApi permite crear un cliente ficticio y obtener un token desde la terminal local, sin agregar rutas de autenticación a esta práctica.

php artisan make:command PrepararDemoApi --no-interaction

php artisan make:test PublicacionesCarritoApiTest --phpunit --no-interaction

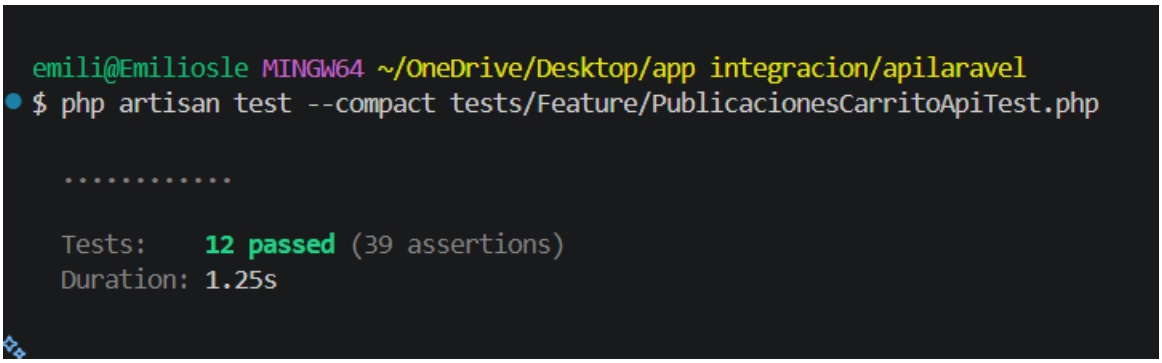
Comprobación de rutas y pruebas

php artisan route:list --path=api --except-vendor

php artisan test --compact tests/Feature/PublicacionesCarritoApiTest.php

Resultado: 4 rutas GET, 12 pruebas aprobadas y 39 aserciones. Las pruebas verifican publicaciones, errores 404, autenticación, separación entre clientes, cálculo de totales, carrito vacío y rechazo de tokens inválidos o caducados.





Ejecución y pruebas en Postman

Preparar el cliente de demostración

php artisan demo:preparar-api

El comando prepara un cliente ficticio y un carrito con dos tazas de USD 8.50 y una caja de regalo de USD 18.00. Muestra un token válido durante 24 horas. Copiar solamente la línea del token. No es necesario registrar una cuenta ni reinstalar paquetes.

Iniciar el servidor

php artisan serve

Dejar abierta la terminal. La dirección habitual es http://127.0.0.1:8000. Si el puerto está ocupado, comprobar si el servidor ya está activo; si Laravel inicia en otro puerto, actualizar base_url en Postman.

Evidencia 1 listado de publicaciones

Método del controlador: PublicacionController@index

Devuelve tres publicaciones de demostración: una novedad, un trabajo realizado y una promoción.

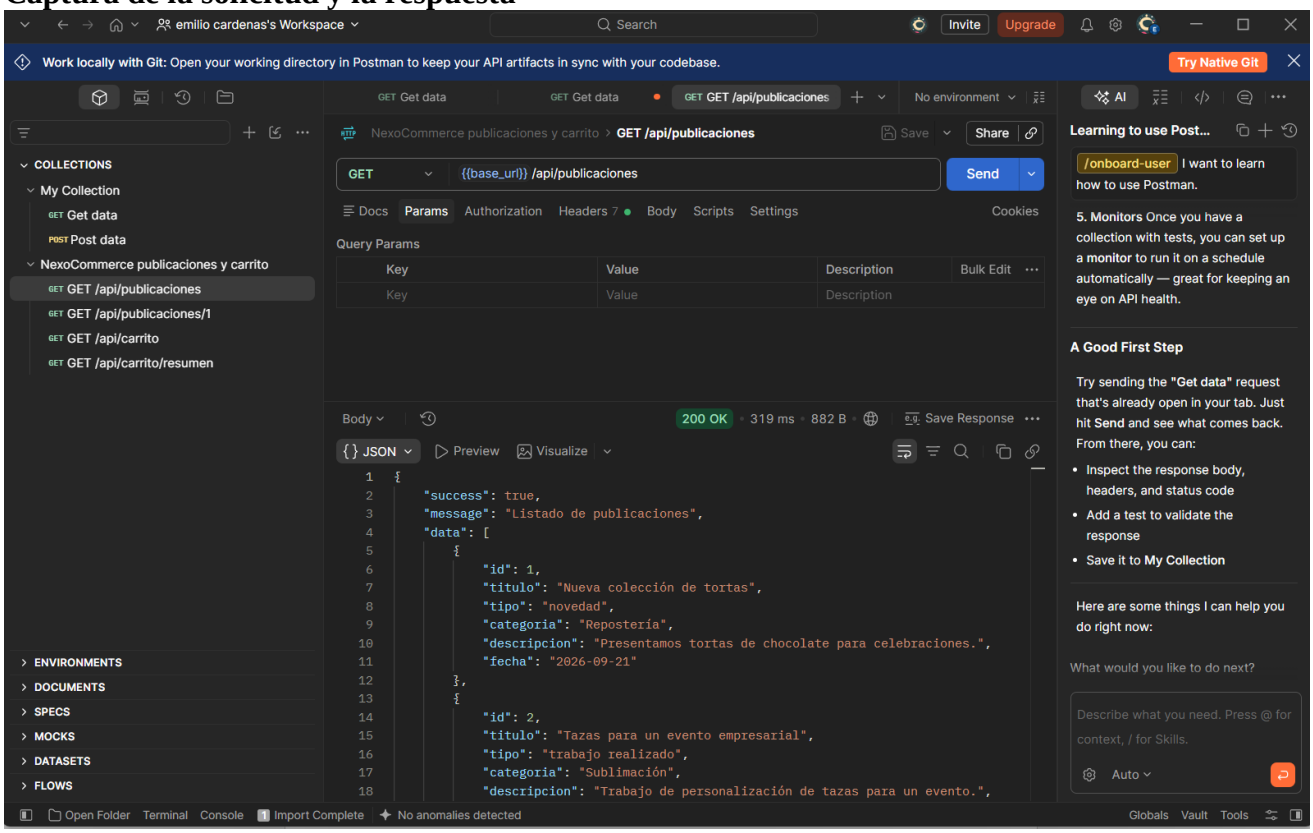
GET http://127.0.0.1:8000/api/publicaciones

Cabecera: Accept: application/json

Sin autenticación.

Estado verificado: 200 OK. Respuesta JSON con success: true.

Captura de la solicitud y la respuesta



Evidencia 2 consulta de una publicación

Método del controlador: PublicacionController@show

Devuelve la publicación con ID 1, titulada Nueva colección de tortas.

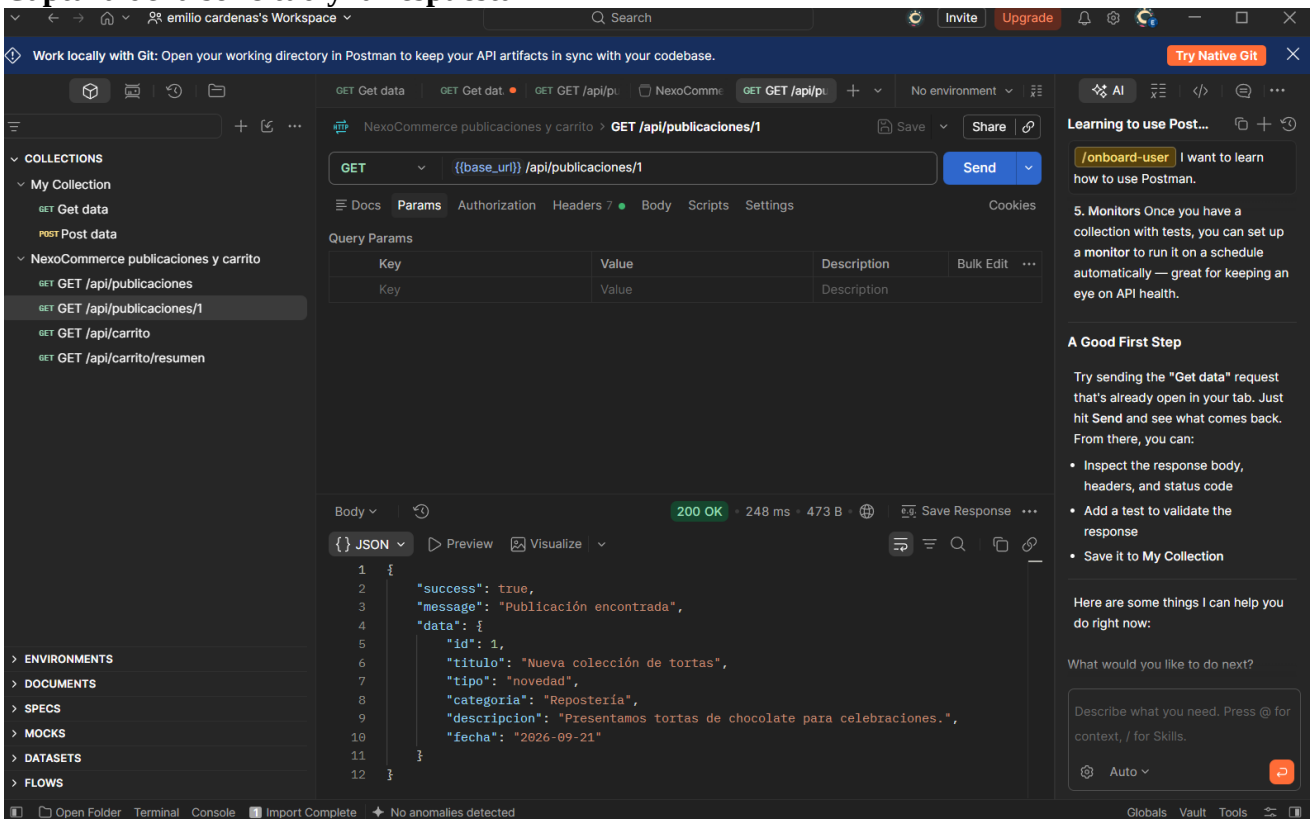
GET http://127.0.0.1:8000/api/publicaciones/1

Cabecera: Accept: application/json

Sin autenticación.

Estado verificado: 200 OK. Respuesta JSON con success: true.

Captura de la solicitud y la respuesta



Evidencia 3 artículos del carrito

Método del controlador: CarritoController@index

Devuelve dos artículos: dos tazas y una caja de regalo. Sus subtotales son USD 17.00 y USD 18.00.

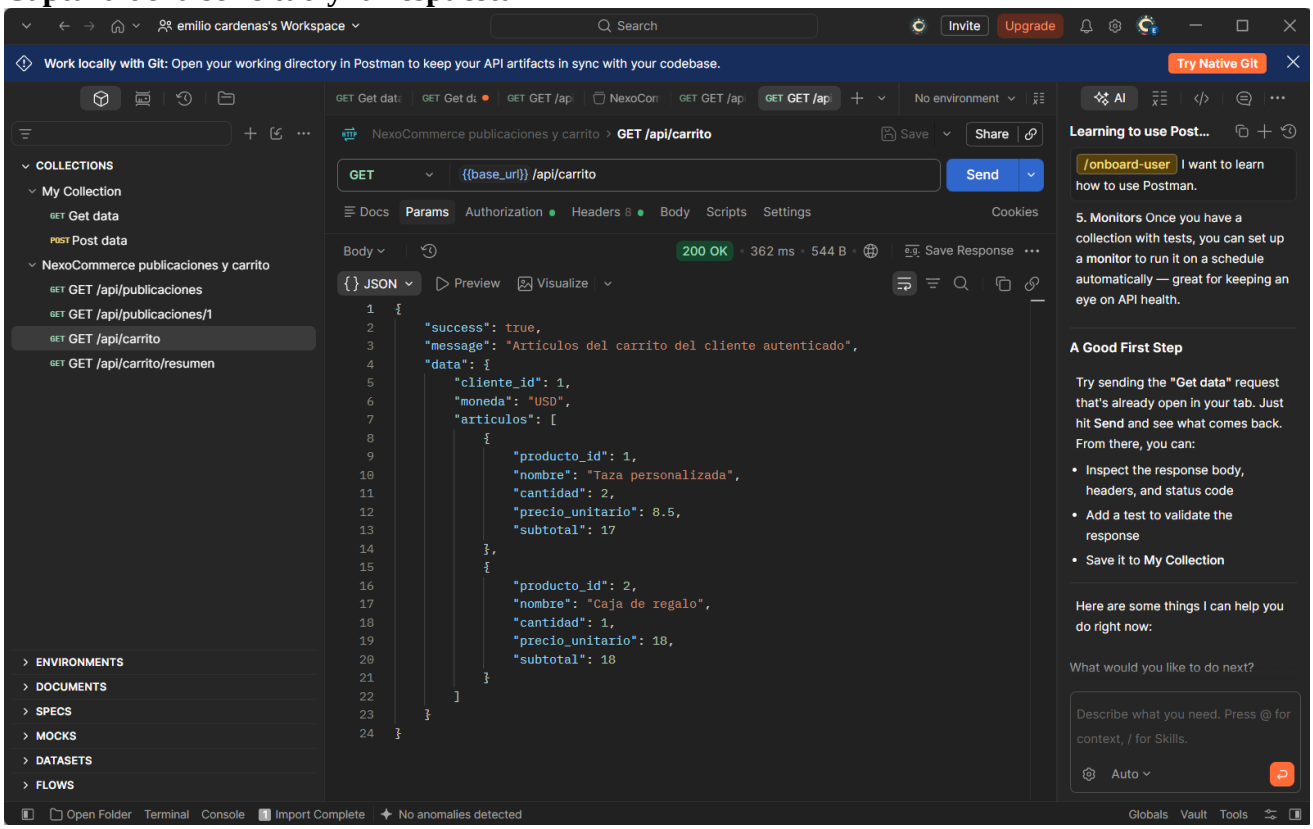
GET http://127.0.0.1:8000/api/carrito

Cabecera: Accept: application/json

Authorization: Bearer Token del cliente de demostración.

Estado verificado: 200 OK. Respuesta JSON con success: true.

Captura de la solicitud y la respuesta



Evidencia 4 resumen del carrito

Método del controlador: CarritoController@resumen

Devuelve 2 productos distintos, 3 unidades en total y un importe de USD 35.00.

GET http://127.0.0.1:8000/api/carrito/resumen

Cabecera: Accept: application/json

Authorization: Bearer Token del cliente de demostración.

Estado verificado: 200 OK. Respuesta JSON con success: true.

Captura de la solicitud y la respuesta

